

NOVAC Internship – Task 4 Daily Documentation

Task: File Upload APIs, MongoDB Integration, Semantic Chunking, and Embedding Storage

Overview

Today's work focused on expanding the AI document ingestion system to support image-based text extraction using OCR (Optical Character Recognition). The previously developed FastAPI ingestion pipeline was upgraded to process image files along with TXT, PDF, and DOCX documents.

The task involved researching OCR technologies, understanding image-to-text extraction workflows, installing OCR dependencies, integrating Tesseract OCR into the FastAPI backend, and processing uploaded images through the existing semantic chunking and embedding pipeline.

By the end of the session, the system was successfully able to upload image files, extract text content from them, perform semantic chunking, generate embeddings, store the processed data in MongoDB, and display the generated chunks dynamically on the frontend.

Resources and References Used

- Tesseract OCR Documentation
- pytesseract Documentation
- EasyOCR Documentation
- Pillow (PIL) Documentation
- FastAPI Official Documentation
- MongoDB Documentation
- Sentence Transformers Documentation

Concepts Learned

OCR (Optical Character Recognition): OCR is a technology used to extract readable text from images. It converts image-based text into machine-readable text that can later be processed using NLP and AI pipelines.

Image-Based Document Processing: Learned how modern AI systems process:

- scanned documents
- screenshots
- handwritten notes
- photographed text documents

through OCR pipelines.

Tesseract OCR: Tesseract OCR was selected for implementation because:

- lightweight setup
- local execution
- offline support
- easy Python integration
- beginner-friendly workflow

EasyOCR: Studied EasyOCR as an alternative deep-learning-based OCR system with:

- multilingual support
- improved recognition accuracy
- neural-network-based text extraction

Multimodal Document Ingestion: Expanded the ingestion system from text-only processing into multimodal processing capable of handling:

- TXT
- PDF

- DOCX
- PNG
- JPG
- JPEG

Libraries, Modules, and Components Learned

pytesseract: Used for extracting text from uploaded image files.

Pillow (PIL): Used for opening and processing image files before OCR extraction.

Tesseract OCR Engine: Used as the backend OCR engine for text recognition.

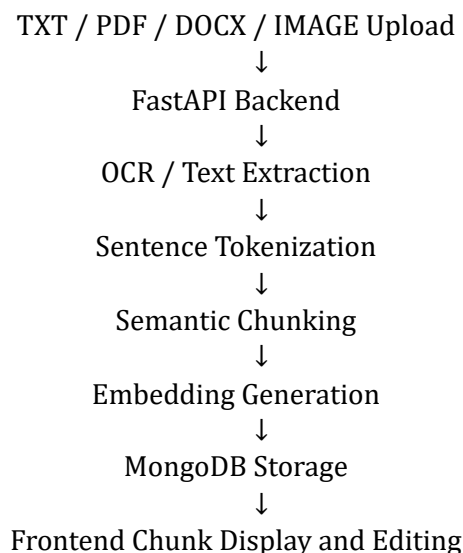
EasyOCR: Studied as an advanced multilingual OCR framework.

OpenCV: Installed as part of OCR preprocessing and image handling dependencies.

Practical Implementation

- Installed Tesseract OCR locally using Homebrew.
- Installed pytesseract and Pillow for Python integration.
- Installed EasyOCR, Torch, and OpenCV dependencies.
- Extended FastAPI backend to support image uploads.
- Added support for:
 - * PNG
 - * JPG
 - * JPEG
- Implemented OCR text extraction using: `pytesseract.image_to_string()`
- Integrated OCR-generated text into the existing semantic chunking pipeline.
- Generated embeddings for OCR-extracted chunks.
- Stored OCR chunks and embeddings inside MongoDB.
- Updated frontend upload support for image file formats.
- Verified successful OCR extraction through frontend chunk display.

Architecture Understood



Challenges Faced

Several dependency and compatibility issues were encountered during installation and setup.

- TensorFlow and protobuf dependency conflicts appeared during OCR-related package installations.
- NumPy version conflicts occurred between OpenCV, Streamlit, and EasyOCR dependencies.
- Tesseract language package setup required additional Homebrew installations.
- Image file handling required separate extraction logic from PDF and DOCX processing.

Resolving these issues improved understanding of:

- Python dependency management
- AI package compatibility
- OCR system setup
- multimodal ingestion workflows

Conclusion

Today's work successfully transformed the existing AI document system into a multimodal ingestion pipeline capable of processing both text-based and image-based documents.

By integrating OCR into the FastAPI backend, the system can now extract text from uploaded images, process the extracted content through semantic chunking and embedding generation, store the processed data in MongoDB, and display editable chunks on the frontend.

The concepts learned today form the foundation of:

- intelligent document processing systems
- OCR-based AI pipelines
- scanned document retrieval systems
- multimodal Retrieval-Augmented Generation (RAG) workflows

Prepared By

Divesh Kumar S